Привет.
Ты - продакт и архитектурный дизайнер с компетенциями в области разработки мобильных приложений под Android (возможно и iOS) с уклоном на веб-разработку.
Помоги создать контекст разрабатываемого продукта.
Накидываю идею.
Мне хотелось бы создать устройство+сервис для моего неговорящего 12-летнего сына с РАС. Он знает буквы и это устройство помогло бы ему печатать что он хочет сказать и показывать нам.
Идея - купить планшет (вероятее всего на Андроиде - цена, больше вариаций и проще общение с файловой системой и настройками) и превратить в устройтсво с автозапускаемым и неубираемым (режим киоска) софтом который, к примеру, сразу делил бы экран горизонтально надвое и снизу была бы клавиатура с буквами рус/англ, пробелом, кнопками очистки, а сверху чистое поле для печатаемого текста.
Я сам разработчик с 25-летним стажем и стеком с nodejs, нативный javascript, es6, веб-верстки, питон-голэнг.
Дальше списком докину сущностей которые я обдумывал, после чего будем обсуждать итеративно
- [ ] Широкий и невысокий (пропорции телефона)
- [ ] Кнопки проговаривания текста
- [ ] Клавиатура максимально как обычная (он привык к планшетным и телевизорным), но можно еще поизучать и посмотреть чем можно точно пожертвовать. например ему не нужны шифты - он привык к написанию заглавных букв
- [ ] Шотбар с кнопками персонажей мультиков (узнаваемые любимые фразы, голоса)
- [ ] Переключатель Рус/англ - он любит канал про поезда, названный латиницей
- [ ] Возможность письма букв пальцем вместо печатания (+ распознавание, смахивание/перечеркивание справа налево для удаления текст и т.д.)
- [ ] Смахивание клавиатуры вниз двумя пальцами до частичного сркытия и поднятие
- [ ] Режим киоска, запуск сразу аппа, кнопок выхода

нет. Выход в обычный режим суперзакрыт (сочетания/последовательности нажатий клавиш для того чтобы родитель вошел настроить вайфай и т.д. вариация - зажимаем какие-то кнопки неудобно для ребенка например "ТОШ" и в нажатом режиме печатаем последовательно еще, скажем, "1", "2", "3" - это будет перехватываемый триггер для открытия панели с входом в обычный дашборд планшета)
- [ ] Чехол-бампер противоударный и с ярким оформлением буквами задней поверхности для того, чтобы ребенок точно отделял это устройство визуально от обычных планшетов, привыкаяя к его четкой функциональной направленности
- [ ] Light/dark режимы
- [ ] Lock ориентации в горизонтальном положении
- [ ] Функция запоминания вводимого (LocalStorage-журналы / отправки на email, tg / ...

Есть и идея далее превратить этот софт-продукт в стор-приложение для помощи другим неговорящим, но это идея навырост.

Привет! Идея очень здравая: ты по сути описал **AAC-девайс (augmentative & alternative communication)**, только без “зоопарка кнопок” на старте — максимально простой “говорящий блокнот” в киоске. Давай зафиксируем **контекст продукта**, чтобы дальше итеративно не расползаться в хотелки и не собрать случайно “комбайн”.

1) Зачем продукт вообще существует

Проблема: ребёнок хочет сообщать желания/мысли, но голосом не может. Быстрый и надёжный “канал связи” снижает фрустрацию у него и у семьи.

Работа, которую продукт делает (JTBD):

- *Когда я хочу что-то сказать, я быстро набираю и показываю текст, не отвлекаясь на настройки и не боясь, что всё сломается/закроется.*

Ключевой критерий успеха: ребёнок сам инициирует коммуникацию через устройство (а не “папа, открой приложение”).

---

2) Пользователи и роли

Основной пользователь

Ребёнок (12 лет, РАС, неговорящий, знает буквы).

- Ценности: предсказуемость, минимальная сложность, мгновенная реакция, “оно всегда работает одинаково”.
- Риски: случайные жесты, закрытия, переключения, “сломал и не вернул”.

Администратор / покупатель

Родитель (ты).

- Ценности: надёжность, контроль, быстрый выход в настройки по “секретному входу”, логи/история, резервное копирование, безопасность.

Вторичные

Учителя/терапевты/родственники — но это “на вырост”, сейчас не грузим.

---

3) Позиционирование (коротко и по делу)

“Киоск-коммуникатор: всегда на экране клавиатура + поле текста + ‘сказать’.”
Никаких меню, никаких “а куда нажать”, никакой возможности выйти случайно.

---

4) MVP: что обязательно, а что позже

MVP (первый релиз — чтобы реально пользоваться каждый день)

1. **Поле текста сверху + клавиатура снизу** (рус/англ, пробел, backspace, очистка).
2. **Кнопка “Сказать”** (TTS) + скорость/голос (минимальные настройки).
3. **Киоск-режим**: автозапуск, блок выхода, блок шторки/кнопок, запрет установки/смены приложений.
4. **Лок ориентации (ландшафт)**.
5. **Большие кнопки/контраст** + light/dark.
6. **Супернадёжное восстановление состояния**: при перезапуске/краше всё возвращается в “режим печати”.

V1 (после того, как MVP “зашёл”)

- **Шотбар фраз/персонажей** (тут важно не превратить в игрушку, но можно).
- **Жесты скрытия клавиатуры** (двумя пальцами — ок, но осторожно: жесты часто становятся “миной”).
- **Рисование букв пальцем + распознавание** (это отдельный мини-продукт, можно позднее).
- **Логи/экспорт** (email/telegram) — полезно, но не должно мешать ребёнку.
- **Профили** (дом/школа), словари быстрых фраз.

Не делаем пока (чтобы не утонуть)

- “Стор-продукт на всех” — пока не поймём, что работает в вашей семье.
- Сложные экраны настроек “как у взрослых AAC” — убьёт простоту.

---

5) Основные сценарии (как продукт живёт)

1. **Быстро сказать**: включил → сразу клавиатура → набрал → показал/нажал “сказать”.
2. **Коммуникация в шуме/на расстоянии**: набрал → крупный текст показывается (возможно кнопка “увеличить текст на весь экран”).
3. **Сбой/перезагрузка**: устройство возвращается в приложение автоматически без действий пользователя.
4. **Родительская настройка**: секретный вход → Wi-Fi/обновления/экспорт → выход обратно.

---

6) UX-принципы (то, что будет держать дизайн в узде)

- **Ноль неожиданных состояний.** Если ребёнок сделал “не то” — приложение должно “само простить”.
- **Одна главная функция на экране.** Печатать/сказать.
- **Большие цели касания** (особенно backspace/очистка — чтобы не попадать случайно).
- **Стабильность раскладки** (никаких прыжков кнопок, никаких “умных” автоисправлений по умолчанию).
- **Предсказуемая обратная связь**: звук клика (опционально), лёгкая анимация, подтверждение “очистить”.

---

7) Технический контекст: как это разумно собрать на Android

Ты веб-разраб, значит хочется WebView. Можно, но с киоском есть нюансы. Варианты:

Вариант A — “надёжно и просто”: нативный Android + встроенная клавиатура

- Делается быстрее, стабильнее, легче с киоск-режимом, TTS и блокировками.
- UI можно сделать “веб-подобным”, но без WebView.

Вариант B — “твой стек”: WebView/Capacitor (или Cordova) + тонкий нативный shell

- Вся верстка/логика на JS/HTML/CSS.
- Нативный слой: kiosk/Device Owner, TTS, автозапуск, блок кнопок, перехват системных событий.
- Риск: WebView + системные ограничения/жесты/фокус ввода — иногда танцы с бубном.

Мой совет как тимлид-прагматик:
Если цель — *быстро получить рабочее устройство для ребёнка*, то **нативный UI на Android** будет менее нервным.
Если цель — *потом быстро масштабировать и переехать на iOS*, тогда **Capacitor + нативные плагины** выглядит логичнее.

Про киоск-режим (важное)

Самый "железобетонный" путь — **Device Owner / Lock Task mode** (обычно через provisioning, QR/ADB, или EMM). Тогда можно реально прикрутить:

- автозапуск,
- запрет выхода,
- блок шторки/кнопок,
- ограничения настроек.

"Просто закрепление экрана (Screen pinning)" — слабее (случайно/намеренно снимается).

---

8) Сущности (то, из чего продукт состоит)

Чтобы тебе было удобно дальше "докидывать список", вот базовая модель:

- **Session** (сеанс набора): текущий текст, язык, состояние клавиатуры.
- **KeyboardLayout**: RU/EN, набор клавиш, размер, спец-кнопки.
- **PhraseButton**: картинка/иконка, подпись, текст, голос/звук, категория.
- **VoiceProfile**: голос TTS, скорость, тон, громкость.
- **ParentalUnlock**: секретная комбинация/тайминг/контекст.
- **LogEntry**: текст, время, событие (набрал/сказал/очистил), экспортировано ли.
- **Theme**: light/dark, контраст, размер шрифта.

---

9) Риски и как их заранее обезвредить

- **Случайная "очистка"** → подтверждение с крупной кнопкой или "удерживать 1 сек".
- **Backspace удержанием удаляет всё** → лучше сначала сделать "удержание = ускоренное удаление", но мягко.

- **Переключение RU/EN случайно** → переключатель должен быть заметным, но не “под пальцем”. Можно “удерживать для переключения”.
- **Жесты** (двумя пальцами/свайпы) → вводить только если реально нужно, иначе это источник багов и хаоса.
- **Шотбар с персонажами превращает всё в пульт мультиков** → лимит и режим “доступно по кнопке”, а не всегда.

---

10) Что делать дальше (без лишних вопросов, но с направлением)

Предлагаю идти итерациями:

1. **Фиксируем MVP-экран**: точные элементы + их размеры/поведение.
2. **Выбираем архитектуру (A или B)** под скорость результата.
3. **План киоска**: какой режим (Device Owner vs pinning), как будешь “прошивать” планшет.
4. **Прототип** (хоть на коленке): чтобы ребёнок потыкал 10 минут — это даст больше данных, чем 3 недели обсуждений.

Если хочешь — кидай следующий список/мысли, а я:

- сведу это в **PRD-lite** (цели, фичи, не-фичи, метрики, риски),
- и параллельно предложу **скелет архитектуры** (модули, хранение, TTS, kiosk, апдейты, экспорт).

И да: ты сейчас на правильном пути — главное, чтобы девайс был “скучным”. Самое лучшее устройство для коммуникации — то, которое не пытается быть интересным 😄

Добавляю в контекст, ты адаптируй его полноту, после формирования сделаем хранимый файл и я далее перейду в интерфейс кодекса для реализации приложения с файлом утвержденного контекеста.
- Сын - Андрюша.
- Он знает буквы, он помнит визуальное написание слов тех предметов и/или действий которые имеют для него вес, значение, интерес.
- Он довольно долгое время практикует бумажные квадратные листочки, боксы с которыми

расположены дома в разных местах вместе с ручками, а также в последнее время пристрастился к магнитному "Эрудиту" с буквами, который периодически берет, чтобы из букв составить слова или фразы. этот пункт важен для того, чтобы когда мы станем формализовать концепт в презентацию (а мы будем это делать для обговарривания идеи с его АВА-куратором) - можно было бы явно указать цепочку преемственности используемых им средств
- Отдельно интересная функция - при наклоне планшеты "от себя" текст который он печатал в верхней части планшета "переворачивается" на 180, чтобы быть читаемым тому, кто стоит напротив.
- Комментарий к: "Ключевой критерий успеха: ребёнок сам инициирует коммуникацию через устройство (а не “папа, открой приложение”)." -- добавишь потом в агент контекста -- Андрюша НЕГОВОРЯЩИЙ, поэтому давай избегать даже примеров про "папа, открой", поскольку это больной вопрос
- к режиму киоска - если будет возможность инициировать его без ввода изначального пароля - будет здорово. если нет - пожертвуем, в прицнипе ему привычно смахивать и печатать давно знакомый ему четырехзначный пин
- запоминание напечатанного для восстановления после краша
- про не превратить в игрушку - правильно, да, посыл функциональности иной, но можно сделать для того чтобы - если делать функцию проговаривания текста (а она может развивательно сказаться - он будет всегда иметь возможность услышать как звучит то что он напечатал даже с ошибками, чтобы понять на слух и исправиться еще не обращаясь к визави) - то проговаривать планшет может одним из привычных голосов. (но тут вероятно будет нужна стилизация голоса, это допник на потом)

про технический контекст. вероятно выбрать вариант А или Б будет сложно не увидев реализацию с нативной клавиатурой андроида, чтобы понять насколько она будет ему удобно (мелко/непривычно и т.д.) и конечно думалось заложиться на постоянный контроль визуально-технического состояния клавиатуры как основы проекта вплоть до функции в родительском контроле обновить ее состояние ихз

репозитория который я буду дорабатывать и релизить отдельно у себя и потом после тестов - на устройстве ребенка.
но конечно и бесконечно танцевать с бубном тоже не хочется, если например какие-то их тех.требований к сервису будут решительно нереализуемый в вебвью - вернемся к решению А/Б

про киоск - да, тут реально важно чтобы никакой случайное шоткатное сочетание его действий не отработало так что покажется дефолт-режим, поскольку он совершенно точно запомнит а) что это возможно и б) возможно запомнит и свои действия для этого, он весьма умен и с очень хорошей памятью. а надо фокусироваться на том, что мы делаем устройство о котором он даже не должен знать, что оно - обычный планшет под капотом с его любимым рутубом.

про риски в числе прочего со свайпами - да, оставим это на последние реализации

докидывай в контекст и формулируем PRD. учти что нам нужен документ для экспорта в сессию кодекса внутри VS, где будет папка проекта, там будет дальше и AGENT.md и TODO.md и CONTEXT.md для погружения в общий текст и онбоардинга в отдельных чат-сессиях

Это уже не “сделать аппку”, это спроектировать для Андрюши устойчивый коммуникационный инструмент с логикой продолжения его текущих практик. И это очень правильный угол атаки: мы не изобретаем новый способ общения — мы **оцифровываем уже работающий паттерн** (бумажные листочки → магнитные буквы → цифровой ввод).

Ниже — структурированный **PRD + CONTEXT.md-контент**, готовый к экспорту в проект. Формат делаю инженерно-сдержанным, без лишней лирики, чтобы его можно было положить в репозиторий как основу.

CONTEXT.md

Проект: Планка (рабочее название)

Цифровой коммуникационный инструмент для неговорящего подростка с РАС.

---

1. Пользователь

Основной пользователь

Имя: Андрюша

Возраст: 12 лет

Особенности:

- Неговорящий.
- Знает буквы.
- Запоминает визуальное написание слов, значимых для него.
- Практикует:
  - Бумажные квадратные листочки с ручками (расположены в разных местах дома).
  - Магнитный “Эрудит” — регулярно составляет слова и фразы.

Важный контекст преемственности

Продукт является логическим продолжением:

1. Письма на бумаге.
2. Составления слов из букв.
3. Визуальной памяти слов.

Мы не внедряем новый способ коммуникации — мы переносим существующий в устойчивую цифровую форму.

---

2. Цель продукта

Создать устройство, которое:

- Всегда готово к печати.
- Не требует запуска.
- Не имеет случайных путей выхода.
- Позволяет быстро набрать текст и показать его.
- Может проговаривать текст вслух.
- Стабильно и предсказуемо.

---

3. Ключевой принцип

Устройство не должно ощущаться как “планшет”.

Это отдельный инструмент коммуникации.

Андрюша не должен воспринимать его как устройство с альтернативными режимами (браузер, видео и т.п.).

---

4. Основной сценарий использования

1. Планшет включается.
2. Автоматически открывается приложение.
3. Экран разделен горизонтально:
   - Верхняя половина — текст.
   - Нижняя половина — клавиатура.
4. Андрюша печатает.
5. Текст можно:
   - Показать.
   - Проговорить.

---

5. Функциональные требования (MVP)

5.1 Основной экран

- Ландшафтная ориентация (заблокирована).
- Верхняя область:
  - Крупный текст.
  - Автоматическое масштабирование.
  - Возможность отобразить текст во весь экран.
- Нижняя область:
  - Клавиатура RU/EN.
  - Пробел.
  - Backspace.
  - Очистка (с защитой от случайного нажатия).
  - Переключатель языка (удержание для смены).

5.2 Проговаривание текста (TTS)

- Кнопка “Сказать”.
- Возможность услышать собственный текст.
- Проговаривание с ошибками (без автоисправления).
- Настройки голоса (в родительском режиме).
- В перспективе — кастомизация голоса (отложено).

5.3 Переворот текста

При наклоне планшета “от себя”:

- Текст автоматически поворачивается на 180°.
- Клавиатура не переворачивается.
- Детектируется по данным гироскопа/акселерометра.

Цель — облегчить чтение стоящему напротив человеку.

5.4 Киоск-режим

Требования:

- Автозапуск при включении.
- Невозможность выхода стандартными кнопками.
- Блокировка шторки и системных жестов.
- Блокировка перехода в другие приложения.
- Минимизация риска случайного открытия системного UI.

Родительский вход:

- Секретная комбинация (например, удержание определённых клавиш + последовательность).
- Альтернатива — PIN (Андрюше знаком 4-значный PIN).

---

6. Надёжность

- Сохранение текущего текста при:
  - Краше приложения.
  - Перезагрузке устройства.
- Локальное хранение состояния.
- Возможность резервного копирования (родительский режим).

---

7. UX-принципы

1. Предсказуемость.
2. Нулевая случайность.
3. Отсутствие “скрытых” состояний.
4. Минимум жестов (свайпы — позже).
5. Никакого автоисправления по умолчанию.
6. Стабильное расположение клавиш (никакой динамики).

---

8. Дополнительные функции (V1+)

- Шотбар с фразами/персонажами (ограниченный).
- Логи сообщений:

    - Хранение локально.
    - Экспорт (email/telegram).
- Родительский контроль обновления клавиатуры из репозитория.
- Light/Dark режим.
- Рисование букв пальцем + распознавание (отложено).
- Кастомизация голосов.

---

9. Технический контекст

Архитектурные варианты

Вариант A — Нативный Android
Плюсы:

- Надёжный kiosk (Device Owner).
- Прямая работа с TTS.
- Минимум слоёв.

  Минусы:
- Два UI-стека, если делать iOS.

Вариант B — WebView + нативный shell (Capacitor)
Плюсы:

- Быстрая разработка.
- Возможность переезда на iOS.
- Контроль UI клавиатуры.

  Минусы:
- Потенциальные сложности с kiosk и системными жестами.

Решение откладывается до прототипирования клавиатуры.

---

10. Критерии успеха

- Андрюша регулярно использует устройство самостоятельно.
- Минимум фрустрации.
- Отсутствие случайных выходов из приложения.
- Устройство воспринимается как отдельный инструмент коммуникации.

TODO.md (начальная версия)

- ☐ Определить целевой планшет (модель, Android версия).
- ☐ Прототип клавиатуры (веб и натив).

- [ ] Проверить удобство стандартной Android-клавиатуры.
- [ ] Исследовать Device Owner provisioning.
- [ ] Реализовать MVP-экран.
- [ ] Добавить TTS.
- [ ] Реализовать сохранение состояния.
- [ ] Реализовать родительский режим.
- [ ] Протестировать с Андрюшей.
- [ ] Зафиксировать наблюдения.

Теперь стратегически важный момент.

Ты правильно чувствуешь риск: если Андрюша обнаружит, что “под капотом” обычный планшет, он может запомнить последовательность действий. У детей с хорошей памятью это реальный сценарий. Поэтому kiosk нужно делать не “косметическим”, а максимально системным.

Следующий шаг я бы предложил такой:

1. Определить конкретную модель планшета.
2. Проверить, возможно ли сделать полноценный Device Owner без MDM.
3. Сделать прототип клавиатуры (быстро — хоть в вебе).
4. Дать Андрюше 10–15 минут теста и просто наблюдать.

После этого мы уже сможем принять инженерное решение A или B без философии и без боли.

Если хочешь — следующим шагом сформируем AGENT.md, который будет задавать правила разработки внутри проекта (архитектура, код-стайл, ограничения, приоритеты надёжности над красотой и т.п.).

все выглядит стройно и правильно.
отдельно есть смысл еще заложиться на то что у нас есть известная свобода в выборе хардварного устройства для этого проекта/продукта. как я упоминал - мне кажется обычный форм-фактор планшета не очень хорош - слишком схожие ширина и высота. а вот форм-фактор мобильного телефона (примерно 2 к 1) но в рамзерах планшета - при разделении на клавиатуру и экран с текстом сделает это устройство максимально использующим

пространство
т.е. отметим что конкретное устройство еще не выбрано и мы можем и должны это учитывать в разработке, ориентируясь на свойства/ТТХ нужные нам в проекте

Вот это уже разговор архитектурного уровня. Ты сейчас не “пишешь приложение под планшет”, ты проектируешь **устройство**. А устройство — это железо + софт + поведенческий паттерн.

И да, ты абсолютно прав: классический планшет 4:3 или даже 16:10 — не оптимален для “экран сверху / клавиатура снизу”. Он квадратный по духу, а нам нужен “коммуникационный билборд”.

Дополнение в CONTEXT.md

11. Аппаратная стратегия

11.1 Устройство не выбрано намеренно

Выбор hardware остаётся открытым.
Проект должен учитывать свободу выбора форм-фактора.

Мы проектируем не под “любой планшет”, а под **оптимальный форм-фактор для коммуникации**.

11.2 Предпочтительный форм-фактор

Желаемые пропорции:
≈ 2:1 (ширина к высоте в ландшафтном положении)

То есть:

- Визуально как смартфон,
- По физическим размерам — как небольшой планшет.

Почему это важно

При разделении экрана:

- Верх — текст (50%)
- Низ — клавиатура (50%)

На “квадратном” планшете:

- Текстовая зона получается слишком низкой.
- Клавиши либо мелкие, либо их слишком мало.

На устройстве 2:1:

- Текст получает полноценную высоту.
- Клавиатура остаётся крупной.
- Пространство используется рационально.

Это улучшает:

- Читаемость для стоящего напротив.
- Скорость печати.
- Визуальную стабильность интерфейса.

11.3 Аппаратные требования (предварительные)

Экран:

- 10–12 дюймов.
- IPS / высокая яркость.
- Хорошие углы обзора.
- Желательно 1920×1080 или выше.
- Соотношение сторон близкое к 18:9, 19:9, 20:9 (в планшетном размере).

Датчики:

- Акселерометр.
- Гироскоп (для функции переворота текста).

Производительность:

- Не критично высокая.
- Важно стабильность и отсутствие лагов.

Память:

- 4 ГБ RAM достаточно.
- 64 ГБ storage достаточно.

ОС:

- Android 10+.
- Возможность Device Owner / Lock Task.

11.4 Концептуальный подход

Мы проектируем UI:

- Без жёсткой привязки к конкретному разрешению.

- С адаптивной системой масштабирования.
- С возможностью “фиксированного” layout под выбранное устройство после финального выбора.

После выбора hardware:

- UI может быть слегка “подточен” под конкретную геометрию.
- Клавиатура может быть оптимизирована под конкретные DPI.

Теперь немного стратегического размышления (рабочая гипотеза, не истина в последней инстанции).

Есть три возможных направления:

1. **Обычный Android-планшет 16:9**
   Простой путь. Самый доступный. Но не идеальный по геометрии.
2. **Большой Android-смартфон + кастомный корпус/рамка**
   Интересный вариант. 20:9 — почти идеал для деления 50/50.
   Минус — физический размер может быть маловат.
3. **Редкие 21:9 планшеты или промо-дисплеи**
   Это уже почти “делаем своё устройство”.
   Дороже, но архитектурно красиво.

Ты сейчас на стадии, когда можно заложить **геометрию как часть продукта**, а не как данность. Это правильный момент.

Теперь важный архитектурный вывод.

Если мы:

- Жёстко контролируем клавиатуру,
- Планируем кастомные размеры,
- И потенциально хотим обновлять её из репозитория,

то использование **системной Android-клавиатуры почти наверняка отпадает**.

Нам нужна своя клавиатура:

- С фиксированной сеткой.
- С управляемыми размерами.
- С предсказуемым поведением.
- Без сюрпризов автоисправления.

А это уже склоняет чашу весов в сторону:

- либо нативного кастомного view,
- либо WebView с полностью своей реализацией.

Ещё одна мысль, которую стоит зафиксировать в голове.

Чем более “специальным” выглядит устройство внешне, тем меньше у Андрюши будет соблазна воспринимать его как обычный планшет.
Яркий бампер с буквами на задней панели — это не косметика. Это якорь идентичности устройства.

Следующий логичный шаг:

1. Зафиксировать желаемую диагональ (в сантиметрах, не в дюймах).
2. Смоделировать 2–3 виртуальных разрешения.
3. Посчитать:
   - высоту текстового поля,
   - высоту клавиш,
   - размер шрифта при чтении с расстояния 1–1.5 м.

И вот тут начинается настоящая инженерия: мы можем сделать мини-расчёт эргономики вместо интуитивного “кажется удобно”.

Если хочешь, следующим шагом можем:

- спроектировать геометрию клавиатуры (количество колонок, размеры),
- или сделать прикидочный расчёт оптимального размера кнопки в мм для 12-летнего ребёнка.

к 11.3 - вполне возможны улучшения ОЗУ-ПЗУ и какие угодно еще, здесь важно что я невероятно люблю своего сына и для него и денег на устройство готов не пожалеть и сделать его, устройство, максимально привычным и удобным на лолгий срок - тоже важный поинт.

докидывай, суммаризуй в виде нового полного контекста, давай его для скачивания, я его добавляю в папку проекта внутри VS и перехожу к обсуждению с тобой же но уже там, чтобы фокусироваться на

формализации и роадмэпа и - с чего наверное мы могли бы начать - с создания полной, полезной, эффектной красочной презентации этого нашего проекта

Контекст готов.

Вот файл для скачивания и добавления в проект:

👉 Скачать CONTEXT.md

Ты сейчас сделал очень правильный архитектурный шаг — зафиксировал не “фичи”, а философию устройства и его место в жизни Андрюши. Это то, что будет удерживать нас от лишних решений в будущем.

Дальше логично:

1. Переносишь файл в проект.
2. Переходишь в VS.
3. Открываем новую сессию.
4. Начинаем с:
   - формирования структуры проекта,
   - архитектурного решения (A/B),
   - и затем делаем **красочную, сильную презентацию** для ABA-куратора — с акцентом на преемственность (листочки → магнитные буквы → цифровой инструмент).

И вот там мы уже сделаем не просто презентацию, а убедительный, продуманный narrative с UX-логикой и доказательной базой.

Погнали дальше. ‹/›

скачал. теперь кинь мне онбоардинг-промпт тоже в виде файла PROMPT.md для инициализации общения в чате, чтобы не просто направить на файл контекста. если ты считаешь что это полезно. если нет - могу просто начать чат там с "прочти контекст из файлы"

Готово.

Вот файл для добавления в проект:

👉 Скачать PROMPT.md

И да — это полезно. Такой промпт:

- экономит первые 10–15 сообщений в новой сессии,
- удерживает фокус на миссии,
- снижает риск “уплыть в красивости”,
- задаёт дисциплину архитектуры.

Когда откроешь новую сессию в VS — просто вставляешь содержимое PROMPT.md и работа сразу начнётся с правильного уровня.

Дальше там логично первым шагом будет:
либо архитектурное решение (A/B),
либо — если хочешь начать красиво и стратегически — делаем презентацию для ABA-куратора как первый артефакт продукта.

Это, кстати, отличный способ самому ещё раз кристаллизовать идею. ‹/›

а есть ли для айфона какое-то кодекс приложение, чтобы я мог видеть чат с кодексом в рамках которого я работаю на десктопе в VS чтобы какие-то результаты видеть и вне компа на мобильнике и мелкие аппрувы делать не подходя к компу ?

Думал на протяжении 8s ›

Коротко: **да, можно “следить за кодекс-работой” с айфона, но это не отдельное iOS-приложение Codex (по крайней мере официально).** Сам Codex как отдельный десктоп-клиент сейчас — **под macOS**. OpenAI +1

Вот реальные варианты, как жить “вне компа” и делать мелкие аппрувы.

Вариант 1 — ChatGPT iOS: Codex прямо внутри (самый близкий к твоему запросу)

OpenAI официально писали, что Codex доступен “даже в ChatGPT iOS app”. OpenAI
То есть: на телефоне ты открываешь **ChatGPT**, заходишь в нужный тред/проект и можешь смотреть результаты, комментировать, давать “ок/не ок”, просить правки.

Плюсы:

- один аккаунт/контекст
- удобно читать и “аппрувить” на ходу

Минусы:

- UX не как в VS (диффы/файлы могут быть менее удобны, чем на десктопе)

Вариант 2 — Codex Web (в Safari)

У Codex есть веб-поверхность (фактически отдельная “Codex on the web”).

OpenAI Help Cen… +1

На айфоне это обычно работает как “почитать/комментнуть”, но полноценный дев-контроль всё равно комфортнее с десктопа.

Вариант 3 — GitHub Mobile + Copilot Chat (если всё в GitHub)

Если ты ведёшь код в GitHub, то для “проверить PR, оставить коммент, принять/завернуть” отлично подходит **GitHub Mobile**.
А ещё там есть **Copilot Chat** (это не Codex, но как мобильный “второй мозг” — норм). GitHub Docs +2

Плюсы:

- лучший мобильный UX для ревью кода, PR, диффов
- можно делать “аппрувы” буквально в лифте

Минусы:

- это про GitHub-флоу, не про “Codex в VS”

Вариант 4 — удалённый доступ к VS/компу (если надо именно “как в VS”)

Если тебе нужно прямо “видеть VS и тыкать кнопки” — самый надёжный путь: **удалённый доступ** (Screen Sharing/AnyDesk/TeamViewer/Tailscale+VNC и т.п.). VS Code Remote Development — это про работу с удалёнными окружениями, но на iPhone VS Code как таковой не живёт официально. Visual Studio Code

Как бы я сделал для твоего кейса (аппрувы с дивана)

1. **Код в GitHub** → PR/Issues
2. **Ревью/аппрувы с iPhone через GitHub Mobile**
3. “Мозг/агент” — **ChatGPT iOS (Codex треды)** для обсуждений, постановки задач и проверки решений

Это даёт максимально удобный мобильный контроль без танцев.

Если хочешь, в следующем сообщении набросаю короткий “workflow”: как разложить репо (PR шаблоны, labels, чеклист релиза), чтобы телефон стал полноценной “панелью управления”, а не просто читалкой.

Источники

такс. оставим пока оповещения, болт с ними
я не хочу в этом контекстном чате, открою новый, но контекст для него очень нужен здешний.
суть - я хочу с твоей помощью начать разработку презентации.
красивой, убеждающей в суперскости этого проекта/продукта/устройства
потребители первые - ABA-тераписты работающие с Андрюшей и его мама.
они долджны будут посмотреть эту пдфку с красивыми картинками и увидеть все что я собираюсь сделать прям с полноцветными превью будущего планшета с клавиатурой, может даже мы сгенерим и фото с Андрюшей сидящим с этим плашетом в руках, в общем нужна покупающая история, хотя здесь речь конечно же не о деньгах.
короче нужна формулировка задачи создания "покупающей" презентации учитываяющая весь здешний контекст, чтобы я скормил это промптом тебе в соседнем чате проекта и там мы сообща пойдем мидджорнить)

Готово.

Вот файл для добавления в проект:

👉 Скачать PROMPT_PRESENTATION.md

Теперь у тебя есть три уровня дисциплины:

- CONTEXT.md — философия и архитектура
- PROMPT.md — инженерный онбоардинг
- PROMPT_PRESENTATION.md — сторителлинг и стратегическая подача

И это уже не просто “идея”, это оформленный проект.

Открывай новый чат в проекте, вставляй PROMPT_PRESENTATION.md — и там мы начнём строить презентацию так, чтобы она не просто объясняла, а давала

ощущение: “Это продумано. Это серьёзно. Это нужно.”